

LLMsVerifier

LLMsVerifier

Verificar. Monitorear. Optimizar.

Resumen

LLMsVerifier es una plataforma de nivel empresarial para verificar, monitorear y optimizar Modelos de Lenguaje Grande (LLM) de múltiples proveedores, construida sobre una prueba de verificación obligatoria llamada “*¿Ves mi código?*”, de modo que solo los modelos que demuestren funcionar correctamente son marcados como utilizables o exportados.

Descripción breve

Plataforma Go que verifica, evalúa, monitorea y optimiza LLM de diversos proveedores. Cada modelo debe superar una prueba obligatoria de visibilidad de código antes de su uso; luego, realiza comprobaciones de latencia, transmisión en tiempo real (*streaming*), llamadas a funciones, visión y *embeddings*, y exporta configuraciones exclusivas de modelos verificados para herramientas AI CLI.

Descripción detallada

LLMsVerifier es una plataforma integral para verificar, monitorear y optimizar el rendimiento de LLM en múltiples proveedores. Su principio fundamental es la *verificación obligatoria*, y no admite concesiones: antes de que un modelo sea marcado como utilizable —o permitido en una configuración exportada—, debe superar de manera fehaciente la prueba “*¿Ves mi código?*”, que realiza llamadas HTTP reales al proveedor y analiza la respuesta en busca de una comprensión genuina, no de un eco verosímil. Un modelo que no pueda demostrar que ve y entiende la entrada simplemente no obtiene la etiqueta de “utilizable”. Una vez superada esta barrera, el Motor de Verificación ejecuta una

batería completa de pruebas de capacidades —existencia, capacidad de respuesta, latencia, *streaming*, llamadas a funciones, visión y embeddings—, y el Motor de Informes convierte los resultados en informes en formato markdown y JSON, listos para tomar medidas.

El sistema es modular y basado en eventos, y ofrece interfaces CLI, TUI, web y REST API sobre un núcleo compuesto por el Motor de Verificación, el Motor de Informes y el Gestor de Configuraciones. Pero no se limita a la verificación. Capas avanzadas incorporan un patrón Supervisor/Trabajador para la descomposición de tareas impulsada por LLM, gestión de contexto mediante ventanas deslizantes y resúmenes con LLM para evitar que sesiones muy largas pierdan coherencia, puntos de control respaldados en la nube y un sistema de conmutación por error con *circuit breakers* y enrutamiento basado en latencia. La infraestructura circundante está diseñada para entornos de producción: un bus de eventos pub/sub, programación mediante *cron*, detección de precios y límites, una base de datos vector para RAG y un sistema de exportación. Una convención de marca distintiva añade el sufijo (*llmsvd*) a cada proveedor/modelo generado, de modo que un resultado verificado sea rastreable a simple vista y nunca pueda confundirse con uno no validado. Además, solo los modelos verificados se incluyen en las configuraciones exportadas para herramientas AI CLI como OpenCode, Crush y Claude Code. La plataforma incluye las herramientas operativas que los equipos necesitan en producción: despliegue con Docker/Kubernetes/Helm, monitoreo con Prometheus/Grafana, autenticación LDAP/SSO y almacenamiento cifrado con SQLCipher.

Por qué lo creamos

Porque la verificación basada únicamente en configuraciones es poco fiable: una clave API puede caducar, un modelo puede quedar obsoleto y un archivo de configuración no ofrece información sobre la latencia real, los errores reales o si el modelo es capaz de ver y comprender realmente la entrada. LLMsVerifier reemplaza el “*está en la configuración, así que debe funcionar*” por pruebas concretas: solo los modelos que demuestran responder correctamente son marcados como utilizables y exportados.

Por qué es un cambio de juego

Convierte a las flotas LLM en *confiables*, un adjetivo que rara vez se gana en un espacio lleno de configuraciones que mienten por omisión. En lugar de esperar que un modelo configurado funcione, los equipos obtienen una garantía verificable y obligatoria de que

cada modelo en uso ha superado pruebas reales, con monitoreo, conmutación por error y exportación exclusiva de modelos verificados cerrando el ciclo desde la validación hasta la producción. Dentro del ecosistema Helix, se convierte en la única fuente de verdad para los metadatos de modelos, proveedores y verificaciones de LLM: otros servicios (incluido HelixTranslate) se enrutan a partir de ella, de modo que toda la plataforma hereda una respuesta honesta a “¿qué modelos funcionan realmente ahora?” en lugar de que cada equipo mantenga su propia suposición optimista.

Qué es innovador

- **Verificación obligatoria de “¿Ves mi código?”** — una compuerta real de comprensión respaldada por HTTP que un modelo debe superar antes de ser utilizable; el diferenciador distintivo del producto y la razón por la que nada no verificado se cuele.
- **Exportación de configuraciones solo verificadas** — las configuraciones generadas para herramientas AI CLI contienen *únicamente* modelos que superaron la verificación, de modo que la configuración que implementas no puede reintroducir silenciosamente un modelo defectuoso.
- **Sistema de sufijo de marca (llmsvd)** — cada proveedor o modelo generado lleva un sufijo rastreable, haciendo visible el origen verificado en todos los lugares a los que llegue el resultado.
- **Detección de capacidades** en múltiples agentes y proveedores CLI — identifica tipos de transmisión (SSE, WebSocket, JSONL, EventStream), compresión y comportamientos de caché en lugar de asumirlos.
- **Conmutación por error resiliente** — interruptores de circuito, enrutamiento basado en latencia que redirige cuando el tiempo hasta el primer token supera un umbral, sondas de salud y división de tráfico ponderada mantienen la flota operativa cuando proveedores individuales fallan.
- **Autonomía de larga duración** — un patrón de descomposición Supervisor/Trabajador junto con puntos de control y integración de memoria permiten sostener sesiones extendidas que, de otro modo, agotarían el contexto.
- **Integración con RAG / vector-DB** para mejorar el contexto fundamentado.

Principales desafíos técnicos y cómo los resolvimos

- **Demostrar que un modelo realmente funciona, no solo que está configurado.** El objetivo central y la parte más

difícil. Se resolvió con la prueba obligatoria de visibilidad de código, que realiza llamadas reales a API y analiza las respuestas en busca de comprensión afirmativa, respaldada por un amplio conjunto de pruebas de capacidades —y luego negándose a exportar cualquier cosa que no haya superado la verificación, de modo que es la prueba, no la configuración, la que controla la producción.

- **Fiabilidad ante múltiples proveedores externos poco confiables.** Se resolvió con un orquestador de conmutación por error que trata la inestabilidad de los proveedores como el caso normal: los interruptores de circuito marcan un proveedor como degradado tras N fallos en M segundos, el enrutamiento basado en latencia evita endpoints lentos, las sondas de salud periódicas verifican la recuperación y el enrutamiento ponderado equilibra modelos económicos con premium.
- **Mantener sesiones autónomas muy largas.** Se resolvió con el patrón de descomposición Supervisor/Trabajador, que divide el trabajo en partes manejables, puntos de control periódicos en almacenamiento en la nube para que el progreso sobreviva a interrupciones y una gestión de contexto en capas (ventana deslizante + resumen de LLM + RAG) para que el modelo mantenga el hilo sin ahogarse en tokens.
- **Proliferación de proveedores.** Se resolvió ocultando numerosos adaptadores Go específicos de cada proveedor tras una interfaz común, con los endpoints reales enumerados de forma centralizada —de modo que añadir un proveedor es un cambio contenido, no una onda expansiva en el código base.

Pila tecnológica

- **Go** — seleccionado como lenguaje base de la plataforma por su concurrencia; impulsa un Motor de Verificación multihilo capaz de sondear múltiples modelos en paralelo, además de los servicios circundantes.
- **Gin** — elegido como servidor REST API, que gestiona autenticación JWT, limitación de tasa y endpoints WebSocket/SSE.
- **SQLite + SQLCipher** — seleccionados para almacenamiento integrado con cifrado a nivel de base de datos, ya que los datos de verificación (claves, resultados) son sensibles y deben cifrarse en reposo por defecto.
- **Redis** — elegido como capa de caché para mantener rápidas las consultas de verificación y metadatos frecuentes.
- **RabbitMQ + Kafka** — seleccionados para impulsar la arquitectura dirigida por eventos: mensajería y transmisión que desacoplan productores de consumidores en toda la plataforma.

- **gRPC + Protocol Buffers** — elegidos para la comunicación fuertemente tipada entre servicios y el transporte de eventos entre componentes.
- **QUIC / HTTP-3 (quic-go)** — seleccionados para soporte de transporte moderno (la documentación del repositorio señala que la disponibilidad de proveedores HTTP/3 es limitada — se trata de una capacidad ofrecida, no una afirmación universal).
- **JWT + LDAP/NTLM** — elegidos para autenticación empresarial, permitiendo que la plataforma se integre con identidades corporativas existentes (SSO/SAML/OIDC mencionados en la documentación).
- **Viper (configuración), Logrus (registros), Brotli/compress (compresión)** — la infraestructura operativa: configuración flexible, registros estructurados y compresión de payloads.
- **Angular** — elegido para la aplicación web de una sola página, la puerta de entrada visual a la verificación y el monitoreo.
- **Python + SDKs JavaScript** — seleccionados para ofrecer a los equipos de clientes acceso de primera clase, documentado mediante OpenAPI/Swagger.
- **Docker, Kubernetes, Helm** — elegidos para el despliegue en producción con monitoreo de salud y escalado automático, de modo que una flota de verificación escale como cualquier servicio moderno.
- **Prometheus + Grafana** — seleccionados para métricas y paneles de control, haciendo que la salud propia de la plataforma sea tan observable como los modelos que supervisa.
- **Testify (Go) + node -test/jsdom (web)** — elegidos para pruebas en capas en el núcleo Go y el frontend web.

Estado y notas de transparencia

- **Estado: beta.** El código fuente de Go implementa verificación HTTP real (un documento heredado que describe la verificación como solo de configuración es aspiracional y obsoleto — el código es la fuente autoritativa).
- **Licencia: por definir.** El archivo README indica MIT, mientras que una etiqueta en el Dockerfile señala Apache-2.0 — resolver antes de publicar.
- Número de proveedores: el README menciona “12 adaptadores”, pero el directorio de proveedores lista aproximadamente 26 — considerar como “12+ / más en desarrollo”. Existen numerosos archivos con estado “FINAL/COMPLETO” aspiracional; el código, la documentación y go.mod son las fuentes autoritativas.
- El repositorio pertenece a la organización vasic-digital, pero funcionalmente actúa como la capa de confianza del clúster de infraestructura Helix LLM.

Nivel de prioridad: Helix-principal (clúster de infraestructura LLM; fuente única de verdad para metadatos de LLM/proveedor/verificación). Se ubica después de HelixTrack.